

Structures In C



Introduction to the structures

A structure in C (or struct) is a **user-defined data type** that groups related variables of **different data types** under a **single name**.

Unlike an array, which requires all elements to be of the exact same type, a structure allows you to **build complex records** (like a student profile with text names, integer IDs, and decimal test scores).

The **struct** keyword is used to define a structure.

Why use Structures?

- To logically group related data.
- It makes your code more organized and easier to read.

Declaration

Syntax:

```
struct tag_name  
{  
    data_type  
    member1;  
  
    data_type  
    member2;  
  
    ...  
};
```

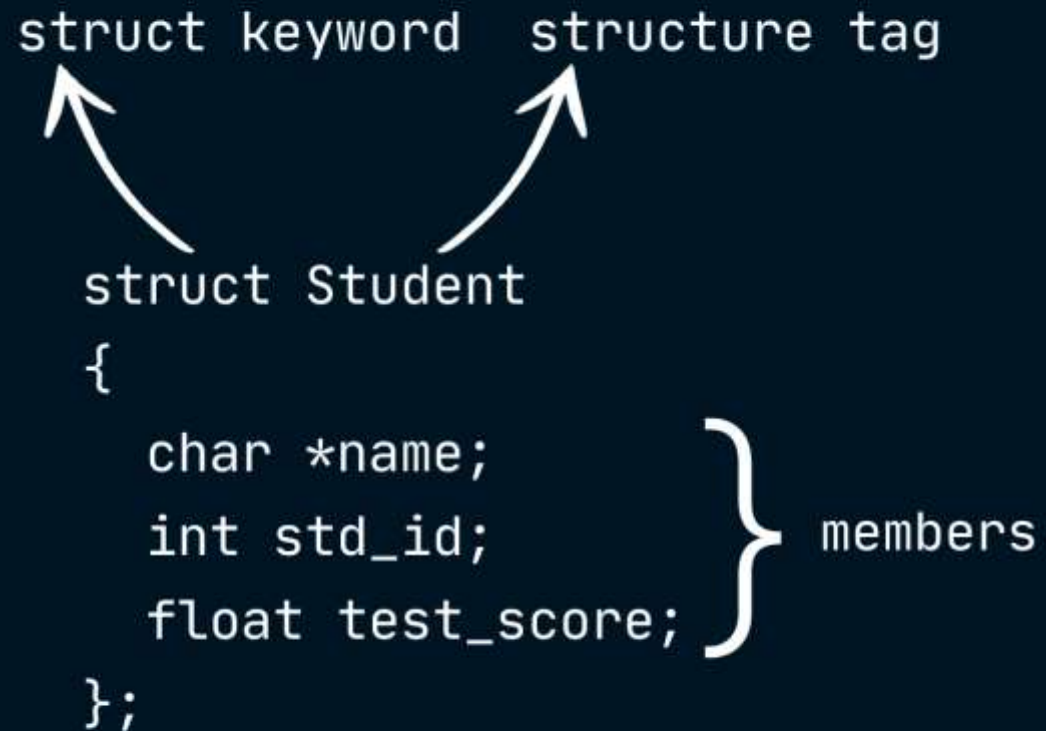
Example:

```
struct Student {  
    char name[50];  
    int studentID;  
    float gpa;  
};
```

How we declare structure

```
struct keyword  structure tag  
    ↖          ↗  
struct Student  
{  
    char *name;  
    int std_id;  
    float test_score;  
};
```

} members



Accessing Structure Members

Syntax: `variable_name.member_name`

You use the dot operator (.) to access individual members of a structure variable.

In the case where we have a pointer to the structure, we can also use the **arrow** operator to access the members.

Function Aspects

Output:

Rahul	20	18.50
Vikas	18	22.00

```
#include <stdio.h>

// Defining a structure
to represent a
student

struct Student {
    char name[50];
    int age;
    float grade;
};
```

```
int main() {

    // Declaring and initializing a structure
    // variable
    struct Student s1 = {"Rahul", 20, 18.5};

    // Designated Initializing another structure
    struct Student s2 = {.age = 18, .name =
        "Vikas", .grade = 22};

    // Accessing structure members
    printf("%s\t%d\t%.2f\n", s1.name, s1.age,
        s1.grade);
    printf("%s\t%d\t%.2f\n", s2.name, s2.age,
        s2.grade);

    return 0;}
```

Copy Structure

Copying structure is simple as copying any other variables. For example, s1 is copied into s2 using **assignment operator**.

```
s2 = s1;
```

```
#include <stdio.h>
#include <stdlib.h>
struct Student {
    int id;
    char grade;
};
int main() {
    struct Student s1 = {1, 'A'};
    // Create a copy of student s1
    struct Student s1c = s1;
    printf("Student 1 ID: %d\n", s1c.id);
    printf("Student 1 Grade: %c", s1c.grade);
    return 0;
}
```

Output:

Student 1 ID: 1

Student 1 Grade: A

typedef

The typedef keyword is used to define an alias for the already existing datatype.

In structures, we have to use the struct keyword along with the structure name to define the variables.

To define some new shorter name for the structure.

// Defining structure

```
typedef struct {  
    int a;
```

```
} str1;
```

// Another way of using typedef with structures

```
typedef struct {  
    int x;  
} str2;
```

typedef

```
int main()
{

    // Creating structure variables using new names

    str1 var1 = { 20 };
    str2 var2 = { 314 };

    printf("var1.a = %d\n", var1.a);
    printf("var2.x = %d\n", var2.x);
    return 0;
}
```

Nested Structures

Structure that contains another structure as one of its members.

```
#include <stdio.h>

struct Student {
    char name[50];
    int studentID;
    struct Date {
        int day;
        int month;
        int year;
    } dob; // dob is the variable
            // for the nested Date
            // structure
};

int main() {
    struct Student student1;

    // Accessing members of the nested structure
    student1.dob.day = 15;
    student1.dob.month = 5;
    student1.dob.year = 2000;

    printf("Student Date of Birth: %d/%d/%d\n",
        student1.dob.day, student1.dob.month,
        student1.dob.year);

    return 0;
}
```

Nested Structures

Structure that contains another structure

```
#include <stdio.h>
```

```
// Date structure is defined separately
```

```
struct Date {  
    int day;  
    int month;  
    int year;  
};
```

```
// Student structure uses the  
previously defined Date  
structure
```

```
struct Student {  
    char name[50];  
    int studentID;  
    struct Date dob; // dob is a  
variable of the Date type  
};
```

```
int main() {
```

```
    struct Student student1;
```

```
    // Accessing members of the separately  
defined structure
```

```
    student1.dob.day = 15;  
    student1.dob.month = 5;  
    student1.dob.year = 2000;
```

```
    printf("Student Date of Birth:  
%d/%d/%d\n", student1.dob.day,  
student1.dob.month, student1.dob.year);
```

```
    return 0;
```

```
}
```

Structure Pointer

A pointer to a structure allows us to access structure members using the **(->)** **arrow operator** instead of the dot operator.

```
#include <stdio.h>

// Structure declaration
struct Point {
    int x, y;
};

int main() {
    struct Point p = { 1, 2 };

    // ptr is a pointer to structure p
    struct Point* ptr = &p;

    // Accessing structure members using structure pointer
    printf("%d %d", ptr->x, ptr->y);

    return 0;
}
```

Structure Examples

Student Record : rollno, name marks

Structure Examples

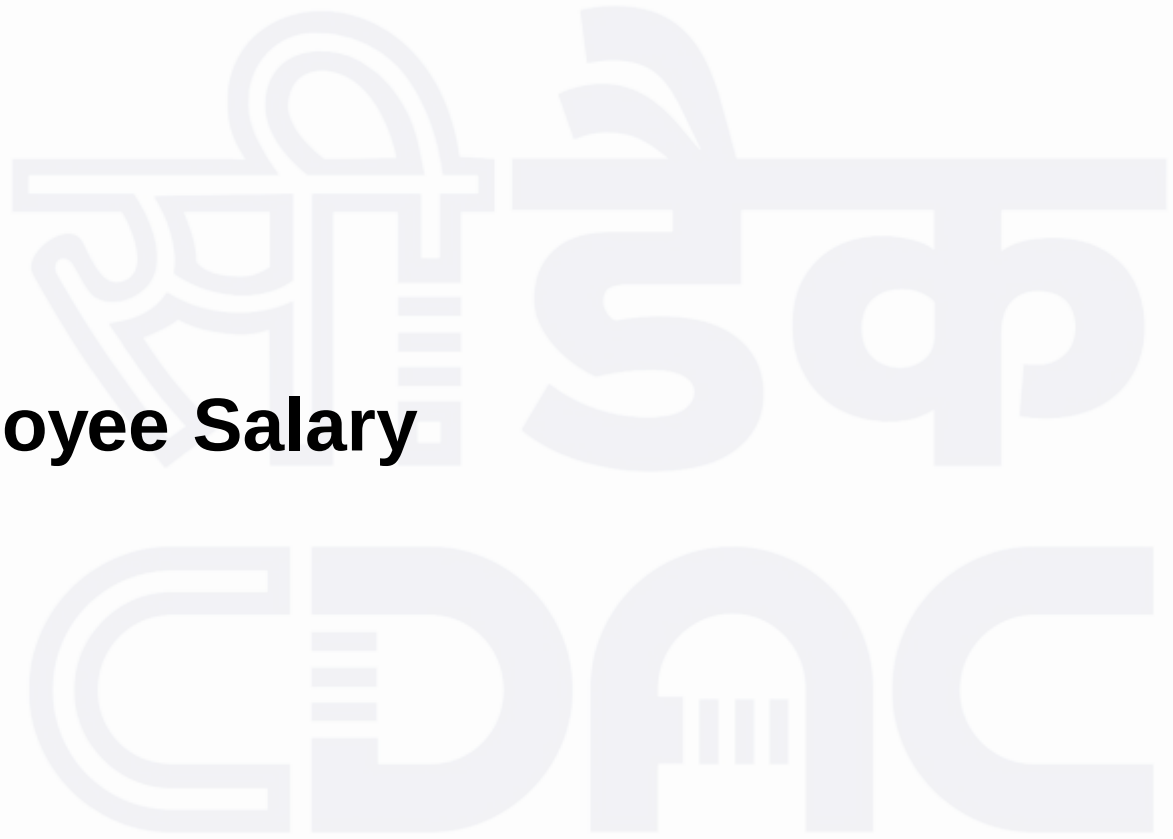
```
#include <stdio.h>

struct Student {
    int roll;
    char name[20];
    float marks; };

int main() {
    struct Student s1;
    printf("Enter Roll No: ");
    scanf("%d", &s1.roll);
    printf("Enter Name: ");
    scanf("%s", s1.name);
    printf("Enter Marks: ");
    scanf("%f", &s1.marks);
    printf("\n--- Student Details ---\n");
    printf("Roll: %d\nName: %s\nMarks: %.2f\n", s1.roll, s1.name, s1.marks);
    return 0; }
```

Structure Examples

Employee Salary



Structure Examples

```
#include <stdio.h>

struct Employee {
    int id;
    char name[30];
    float salary;
};

int main() {
    struct Employee e = {101, "Aleena", 60000.50};
    printf("Employee Details:\n");
    printf("ID: %d\n", e.id);
    printf("Name: %s\n", e.name);
    printf("Salary: %.2f\n", e.salary);
    return 0;
}
```

Structure Examples

Array of Structures (Multiple Students)

Structure Examples

Array of Structures (Multiple Students)

```
#include <stdio.h>
```

```
struct Student {  
    int roll;  
    char name[20];  
    float marks;  
};
```

```
int main() {  
    struct Student s[3];  
    int i;
```

```
    // Input
```

```
    for (i = 0; i < 3; i++) {  
        printf("Enter Roll, Name, Marks for Student %d: ", i+1);  
        scanf("%d %s %f", &s[i].roll, s[i].name, &s[i].marks);  
    }
```

```
    // Output
```

```
    printf("\n--- Student Records ---\n");  
    for (i = 0; i < 3; i++) {  
        printf("Roll: %d | Name: %s | Marks: %.2f\n", s[i].roll, s[i].name, s[i].marks);  
    }
```

```
    return 0;
```

```
}
```

Structure Examples

Structure inside Function

Structure Examples

```
#include <stdio.h>

struct Point {
    int x, y;
};

//function declaration (prototype)
void printPoint(struct Point);

int main() {
    struct Point p1 = {10, 20};
    printPoint(p1);
    return 0;
}

void printPoint(struct Point p) {
    printf("Point = (%d, %d)\n", p.x, p.y);
}
```

Structure Examples

Structure to a function

```
#include <stdio.h>

struct student {
    char name[25];
    int rollno;
    int mark;
};

void printstr(struct student);

int main()
{
    struct student s1 = {"John", 17, 92};
    printstr(s1);
    return 0;
}

void printstr(struct student s)
{
    printf("\n Name: %s", s.name);
    printf("\n RollNo: %d", s.rollno);
    printf("\n Mark: %d", s.mark);
}
```

Structure Examples

Structure to a function by **pointer** (rather than by value)

```
#include <stdio.h>

struct student {
    char name[25];
    int rollno;
    int mark;
};

void printstr(struct student*);

int main()
{
    struct student s1 = {"John", 17, 92};
    struct student *sp;
    sp = &s1;
    printstr(sp);
    return 0;
}

void printstr(struct student *p)
{
    printf("\n Name: %s", p->name);
    printf("\n RollNo: %d", p->rollno);
    printf("\n Mark: %d", p->mark);
}
```

Structure Examples

These examples cover:

- Single structure variable
- Initialization
- Array of structures
- Passing structures to functions

Task

Student Management System

(Using Structures)

Features:

1. Add student
2. Display all students
3. Search student by roll number
4. Exit